Week 04 Journal Activity

Algorithm Used

-By itself, the Cipher Block Chaining (CBC) mode increases encryption security by providing interdependent blocks. On attempting Internet transmission it introduces uniqueness by giving no two distinct ciphertexts from the same plaintext and IV equal in length. XORing the IV against the plaintext block and then encrypt the result with the key, is what the encryption part begins with. Each subsequent block is XORed with the last ciphertext, making a chaining to prevent the deciphering of the ciphertext by pattern recognition. That means to decrypt, decrypt using the key and XOR with the previous ciphertext or IV. Improving confidentiality comes from making it impossible for identical plaintext blocks to produce the same ciphertext. IV management care should be practiced so as not to fall prey to reuse pitfalls that may compromise encryption. More importantly, errors can cascade during decryption.

Perform XOR bit by bit using the rules:

0 ⊕ 0 = 0

0 ⊕ 1 = 1

1 ⊕ 0 = 1

1 ⊕ 1 = 0 ✅ Final Answer: 101101

Encrypt a Random 5-bit Plaintext using a Random 3-bit Key

Plaintext (P) = 01100

Key (K) = 101

Ciphertext (C) = 00000

Decrypt the Ciphertext using a Random 3-bit Key

Ciphertext (C) = 11111

Key (K) = 110

Plaintext (P) = 01110

SBC in CBC Mode

Python code created

SBC Lookup Table (Example)

run: nano sbc_cbc.py

sbc_table = { "00000": ["10101", "11000", "00111", "01010", "01111", "10000", "11101", "00011"], "00001": ["00100", "11110", "10001", "01100", "11010", "10111", "00001", "01111"], "10001": ["00011", "11001", "01101", "10000", "01011", "10110", "11011", "11100"], "10110": ["11100", "01011", "10100", "00010", "00101", "11101", "10011", "11000"] }

def xor(a, b): return "".join(str(int(x) ^ int(y)) for x, y in zip(a, b))

def sbc_encrypt(plaintext, key): return sbc_table.get(plaintext, ["0"] * 8)[int(key, 2)]

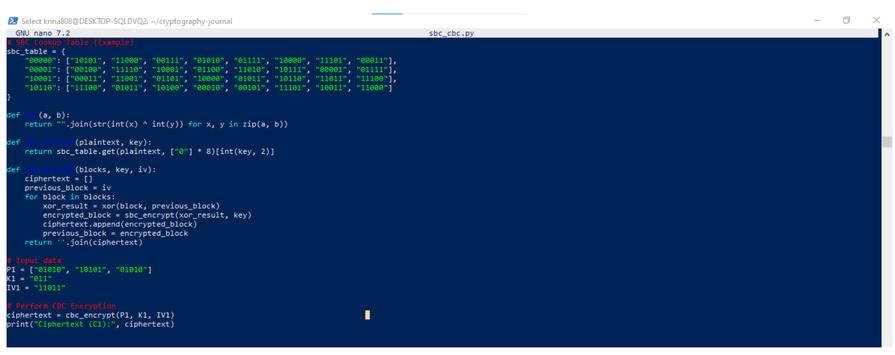
def cbc_encrypt(blocks, key, iv): ciphertext = [] previous_block = iv for block in blocks: xor_result = xor(block, previous_block) encrypted_block = sbc_encrypt(xor_result, key) ciphertext.append(encrypted_block) previous_block = encrypted_block return ''.join(ciphertext)

Input data

P1 = ["01010", "10101", "01010"] K1 = "011" IV1 = "11011"

Perform CBC Encryption

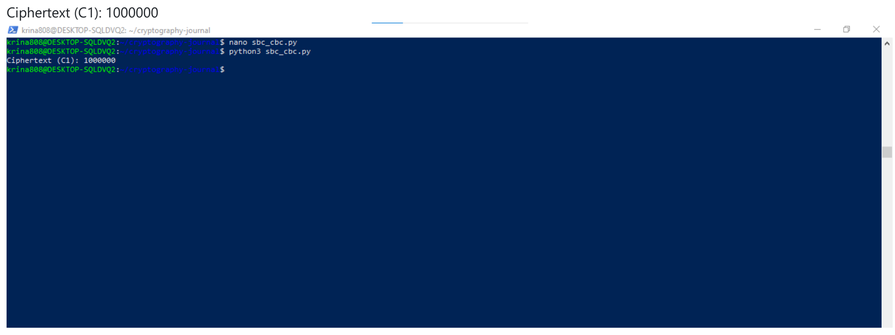
ciphertext = cbc_encrypt(P1, K1, IV1) print("Ciphertext (C1):", ciphertext)



Then run: python3 sbc_cbc.py

Outut:

Ciphertext (C1): 1000000



Attacks on Systems:

This week's activity taught us that CBC mode offers higher security than ECB owing to lack of pattern recognition in ciphertext. The encryption is however subject to attacks such as IV reuse or poor key management. In CBC mode, we reuse the same IV and have identical ciphertexts for the blablabla of the same plaintext, giving attackers a hint regarding patterns.

Difficulties in Understanding Security Systems:

However, the importance of IV management in CBC mode was something that I could not make sense of. It turned out that strong encryption alone would not prevent complete decryption when IV reuse or poor randomization is possible.

Links to Security Systems:

OWASP Cryptography Cheat Sheet

Link: OWASP Cryptography Cheat Sheet

Owasp has a great guide with best practices and pitfalls for implementing cryptography, including key management, algorithms, avoiding common blunders such as failure to properly manage IV.

Insights & Reflections:

-For this activity I tuckered out a simple block cipher (SBC) in Cipher Block Chaining (CBC) mode to encrypt the plaintext. CBC mode introduced another way to provide additional security in case of identical blocks of plaintext by requiring use of an initialization vector (IV). In this way the patterns within encrypted data where otherwise possible are in this effectively mitigated, as in, for example, ECB. Implementing SBC I've noted how the XOR operations are used during the processes of encryption and chaining, the operations are based on the common term - the operation of XOR depends on both the plaintext between two ciphertext blocks. This induces diffusion and strengthens security of the encryption. Moreover, the restricted use of a small block size and insecure key generation were also highlighted as the inadequacies of SBC for genuine applications, therefore, they argued the necessity of using a more robust algorithms such as AES. In adding to the hands-on approach, it further helped me understand the relevance of the principle of randomness behind IV selection as well as key management struggles. Through this exercise I gained knowledge as to how encryption algorithms work in real life to ensure the security of sensitive information.